

Parte 1

Patrones y Principios de Desarrollo de Software

Grupo 3

Pedro Caso, Diego Calderon, Miguel Rosas, Hugo Mendez, Javier Alvarado.

Cliente-Servidor:

- Problema que resuelve:

Resuelve el desorden y la descentralización de recursos, evitando una limitación en el uso del sistema y una lógica poco funcional. Lo resuelve mediante el orden y organización de los sistemas, indicando claramente como se comunican dichos sistemas entre sí.

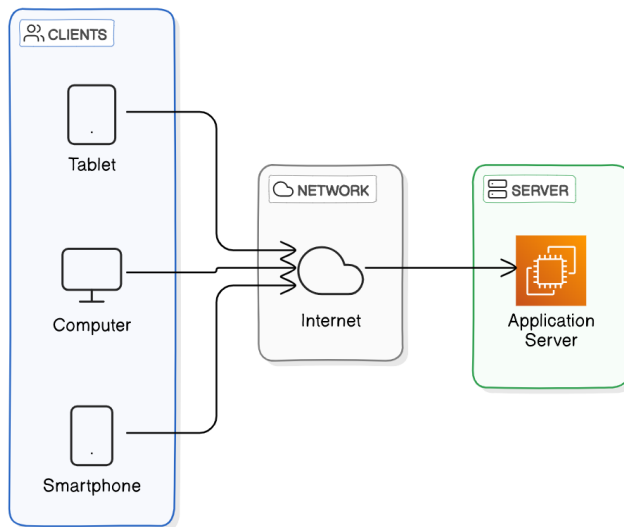
- Estructura general:

Su estructura general se basa en como los sistemas tienen distintas funcionalidades y propósitos, por esa razón se divide de la siguiente manera:

1. Cliente: es el que realiza las peticiones, espera la respuesta del server y ve los resultados obtenidos.
2. Servidor: es el encargado de recibir solicitudes y de gestionar una respuesta con los datos almacenados.
3. Protocolo de comunicación: Es el que define como se habla el cliente con el servidor, como por ejemplo HTTP, HTTPS, TCP, etc.

- Diagrama:

Simplified Client-Server Architecture



- Ventajas y desventajas:

Entre las ventajas se encuentran:

- Centraliza la información
- Es escalable
- efectúa una simplicidad en la estructura
- Soporte de muchos usuarios

Entre las desventajas se encuentran:

- El servidor puede caerse y no existe comunicación en ese intervalo de tiempo.
- Puede provocar latencia de red.
- Puede provocar cuello de botella.

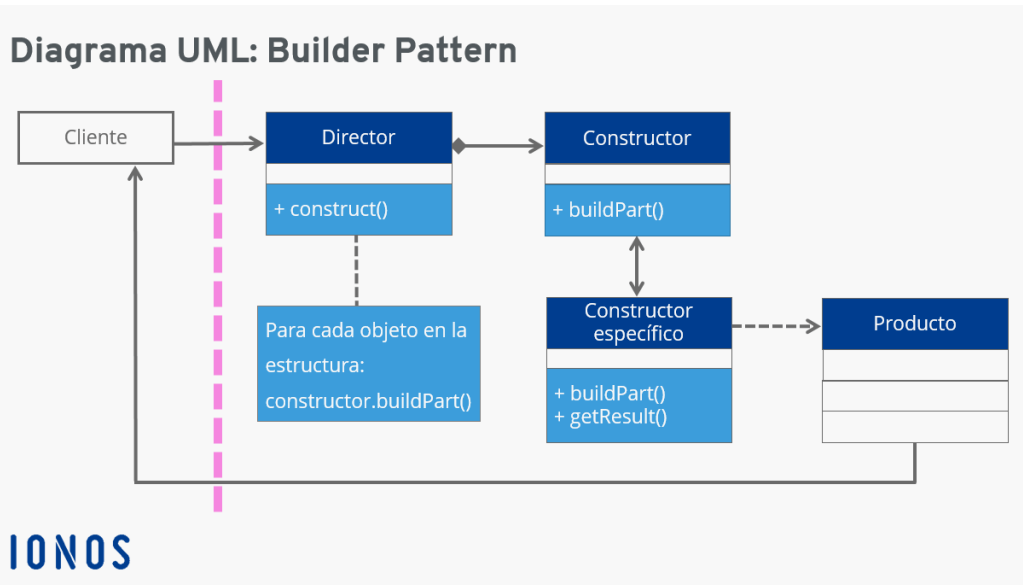
- Principios de diseño:

- Principio de responsabilidad única: cada una de las partes del sistema se encarga únicamente de una tarea en específico, lo que permite un orden y estructura clara y manejable.
- Separación de intereses: separa de manera clara la interfaz del cliente y la de la base de datos del servidor.
- Encapsulamiento: se protegen los datos del servidor y de la base de datos
- Abstracción: El cliente no necesita conocimientos para utilizar el servidor, solamente realiza peticiones.

- Justificación:
El patron cliente-servidor permite administrar de muy buena manera los recursos además de facilitar la escalabilidad del sistema, acceso remoto, adaptabilidad a las últimas tecnologías, por lo que es una solución muy común y fácil de usar para proyectos que requieran manejo de datos de forma interna y un acceso fácil para el usuario mediante peticiones. Sin embargo, se debe estructurar de buena manera para evitar el cuello de botella o latencia en la red.

Builder:

- Problema que resuelve:
Al momento de iniciar un proyecto se tienen muchos parámetros en la mayoría de las ocasiones, haciendo el código complejo y muy propenso a errores debido a la repetición de código y lógica mal codificada. El builder logra solucionar este problema mediante la construcción de objetos complejos paso a paso cuando se tiene demasiados atributos o configuraciones.
- Estructura general:
 - Builder: se encarga de definir los pasos a seguir para la construcción de las partes del producto.
 - ConcreteBuilder: se encarga de implementar la interfaz y proporciona los detalles de la construcción para un tipo de producto en específico.
 - Director: se encarga de realizar las llamadas de como se llaman los elementos para un resultado estándar, este es opcional dependiendo de donde se trabaje.
 - Producto: es el producto final que se construye y se plantea desde el inicio, ya que contiene múltiples atributos y elementos para su construcción.
 - Cliente: utiliza el builder para construir el objeto y se encarga de ver los resultados finales de la construcción del producto.
- Diagrama:



- Ventajas y desventajas:

Ventajas	Desventajas
• Fácil creación de objetos • Mejor legibilidad del código • Evita constructores gigantes • Permite representar el mismo objeto de diferentes formas	• Aumenta la cantidad de código • Puede ser innecesario en muchos casos • Mayor complejidad

- Principios de diseño:

- Principio de responsabilidad única: La clase principal representa el objeto y el builder se encarga únicamente de construirlo, separando qué es el objeto de cómo se construye
- Principio Abierto/Cerrado: Permite extender la forma en que se construyen objetos sin modificar la clase original
- Encapsulamiento: El proceso de construcción queda oculto dentro del builder, el usuario del objeto no necesita saber cómo se construyó.
- Bajo acoplamiento: El código que usa el objeto no depende de constructores complejos ni de múltiples combinaciones de parámetros, lo que reduce errores y dependencias innecesarias
- Separación de construcción y representación: El Builder separa el proceso de creación del objeto final. El mismo proceso puede generar distintas configuraciones del objeto.

- Justificación:

- El uso de patrón builder se justifica por la necesidad de separar la construcción de un objeto complejo de su representación, permitiendo que el mismo proceso de construcción pueda crear diferentes representaciones. Evita el problema del constructor telescópico, donde un programador se enfrenta a constructores con una lista interminable de parámetros.

Composite

- Problema que resuelve:

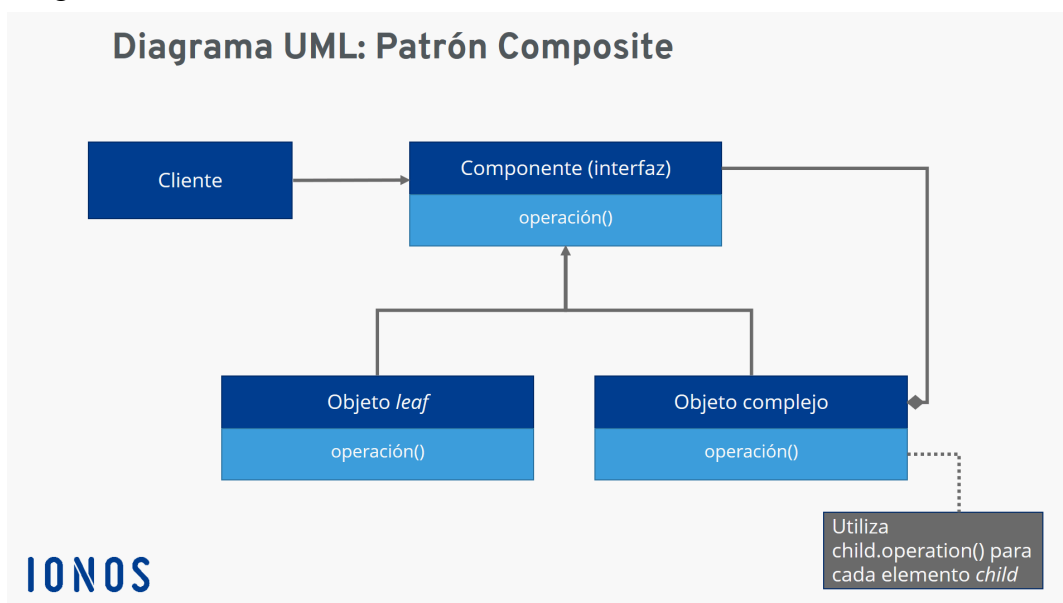
El problema que este resuelve es la falta de uniformidad, permitiendo que el cliente ignore la diferencia entre composiciones de objetos y objetos individuales, reduce el acoplamiento y fragilidad del código ante cambios en la estructura del servidor.

- Estructura general:

Esta se basa en una estructura de recursividad y el polimorfismo

- Componente: Define la firma de las operaciones comunes para los objetos simples y complejos.
- Hoja: Es el elemento básico que no tiene hijos. Implementa la lógica de negocio final.
- Compuesto: El contenedor que almacena la lista de hijos (hojas u otros compuestos). Implementa los métodos de la interfaz delegando el trabajo a sus hijos.
- Cliente: Este es el que interactúa con todos los elementos a través de la interfaz del componente.

- Diagrama:



- Ventajas y desventajas:

Ventajas:

- Este es transparente, ya que el cliente puede tratar estructuras complejas como si fueran objetos únicos.
- Es escalable, ya que es fácil de añadir nuevos tipos de elementos sin alterar el código existente.
- Tiene buena simplificación, ya que elimina la necesidad de utilizar múltiples sentencias if o switch para poder verificar los tipos de datos.

Desventajas:

- Es difícil restringir que componentes pueden añadirse a otros a la hora de realizar una compilación.
- La recursividad profunda puede llegar a dificultar el rastreo de errores en el flujo de datos.
- Estructuras masivas pueden llegar a consumir recursos si no se realiza una optimización en el recorrido.

- Principios de diseño

- Principio de Responsabilidad Única: Separa la gestión de la jerarquía de la lógica operativa.
- Principio de Abierto/Cerrado: Permite extender el sistema con nuevas clases de hojas o contenedores sin modificar la interfaz establecida.
- Principio de Inversión de Dependencia: El cliente depende de la abstracción, no de implementaciones concretas del servidor.
- Principio de Sustitución de Liskov: Un componente puede ser usado en cualquier lugar donde se espera un componente sin romper la funcionalidad

- Justificación:

El patrón Composite es la solución adecuada en el modelo cliente-servidor, ya que permite construir sistemas escalables y mantenibles. Garantiza que el servidor envíe respuestas que el cliente pueda procesar, también al evitar que el cliente deba conocer la estructura interna del servidor hace una reducción de dudas técnicas y facilita que el backend evolucione sin obligar a actualizar el código frontend.

State:

Problema que resuelve:

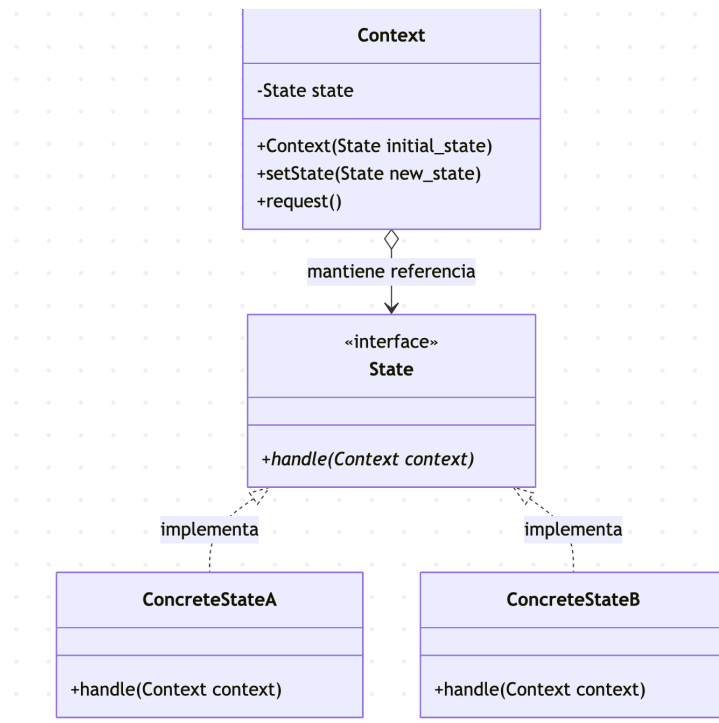
State resuelve el problema de tener el código lleno de if-else y switch para hacer la evaluación del estado en el que se encuentra el objeto previo a ejecutar una acción. State encapsula el comportamiento de cada estado dentro de las diferentes clases.

Estructura general:

State se divide en 3 partes principales:

- Contexto: Esta es la clase principal con la que interactúa el cliente. Tiene una referencia a un objeto tipo State que representa el estado actual del contexto.
- Estado: Esta es una interfaz o una abstract class que declara los métodos de todos los estados concretos que implementan.
- Estados Concretos: Estos son los que implementan la interfaz Estado. Cada uno de estos proporciona su propia implementación para los métodos declarados y estos definen cómo se debe de comportar el Contexto cuando se encuentra en un estado en específico.

Diagrama:



Ventajas y desventajas:

- **Ventajas:**
 - Deja un código limpio quitando las sentencias condicionales largas asociadas a la máquina de estados.
 - El código de un estado específico se encuentra en un solo lugar que es su propia clase.
 - Hace que la transición de estados sea más fácil.
- **Desventajas:**
 - Si el objetivo del programa es solamente tener pocos estados y los cuales no cambian de estado tan frecuente, el State puede llegar a ser excesivo y no se aprovecha de la mejor manera.
 - Hace que existan más clases de manera significativa dentro del proyecto.

Principios de diseño:

- State favorece el principio de responsabilidad única (SRP) ya que este patrón de diseño organiza el proyecto de manera que cada estado concreto tenga su propia clase con una sola responsabilidad que es manejar la lógica exclusiva de ese estado. Gracias a esto el contexto no se debe preocupar de cómo se comporta el estado, solo delega esta responsabilidad.
- State favorece el principio de abierto/cerrado (OCP) porque permite introducir nuevos estados al sistema con la creación de nuevas clases para dichos estados concretos sin la necesidad de alterar el código de la clase Contexto ni de los estados que ya existen.

Justificación:

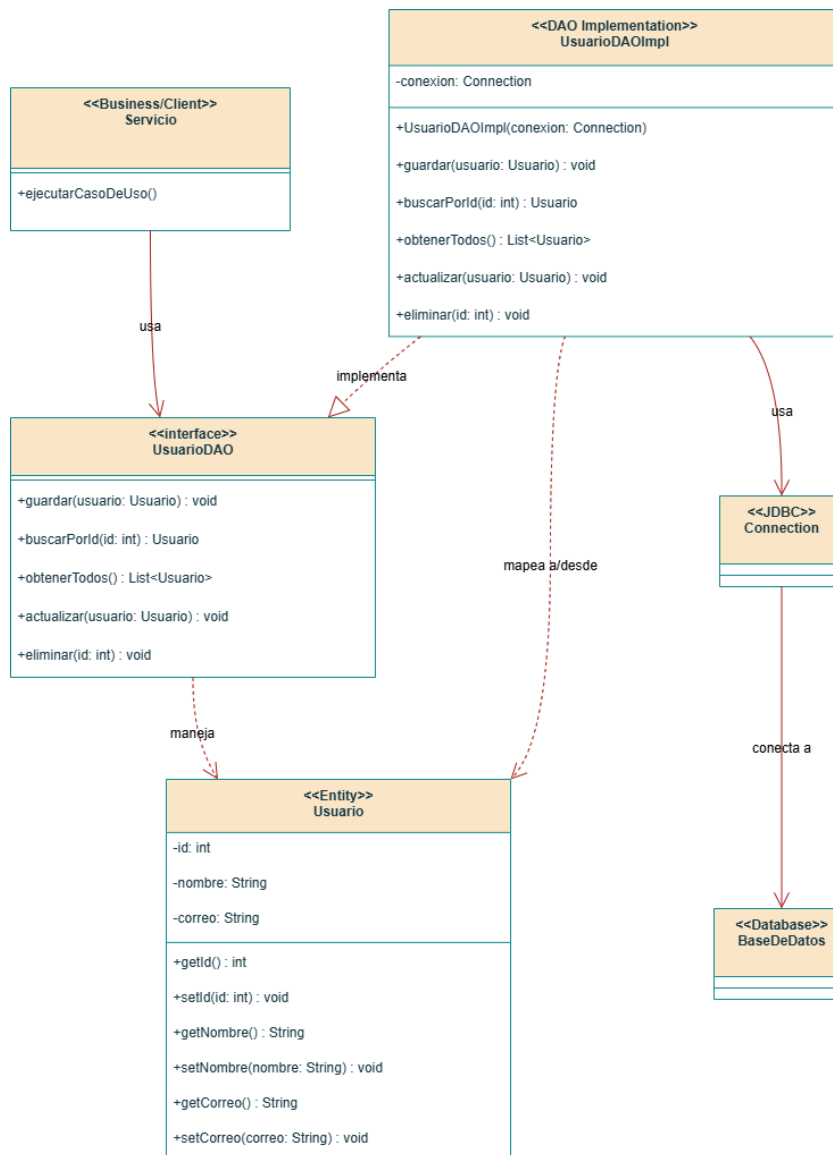
El patrón de diseño State es una solución adecuada para el patrón arquitectónico cliente-servidor porque evita la complejidad de las sentencias condicionales largas y las reemplaza con un polimorfismo puro que delega el comportamiento de estados específicos. Al encapsular los comportamientos de los estados en su propia clase se garantiza las transiciones de estado seguras y atómicas ya se aísla la lógica de cada estado para que cumpla con el principio de responsabilidad única.

DAO:

- Problema que resuelve: Es un patrón que separa la lógica del acceso a la base de datos, haciendo que el código sea más fácil de mantener y logrando que todo el código SQL exista solo en una clase específica, sin mezclar con el resto del código
- Estructura general y componentes: Consta de 4 componentes principales:
 - Entidad: Representa la tabla de la base de datos
 - Interfaz DAO: Aquí se definen las operaciones que se podrán hacer, como obtener, eliminar, modificar, etc.
 - Implementación: Es donde va el código real que accede a la base de datos.
 - Cliente: Es quien utiliza el DAO, quien solo llama los métodos sin tener que saber SQL.

-

Diagrama:



- **Ventajas y desventajas:**

Ventajas	Desventajas
• Fácil mantenimiento • Mejor testeo • Mayor escalabilidad • Separación de responsabilidades	• Puede quedar obsoleto • Mayor número de clases • Puede ser repetitivo • No modela bien lógica compleja

- **Principios de diseño:**

- Principio de responsabilidad única: Cada clase debe tener una sola razón para cambiar, si la cambia la base de datos entonces cambia el DAO nada más, si cambian las reglas del negocio entonces cambia esa capa únicamente.

- Principio de inversión de dependencias: Ninguna clase debe depender de implementaciones concretas, si no de implementaciones abstractas.
- Encapsulamiento: El acceso a la base de datos está encapsulado
- Bajo acoplamiento: Las capas están poco acopladas, lo que hace el sistema más flexible, ya que la lógica no depende de SQL y el sistema no depende directamente de la base de datos
- Alta cohesión: Todo lo relacionado con persistencia está en un mismo lugar
- Justificación: El patrón DAO se justifica porque ayuda a mantener el código ordenado y fácil de entender, separando claramente la parte que maneja la base de datos de la parte que contiene las reglas del sistema. Esto evita mezclar consultas con la lógica principal, lo que hace que el programa sea más limpio y sencillo de mantener. Además, si en el futuro cambia la base de datos o la forma en que se guardan los datos, solo es necesario modificar esa capa sin afectar todo el proyecto. En general, DAO aporta organización, claridad y mayor facilidad para hacer cambios sin romper el sistema.

Minicaso Integrado de Aplicación

Sistema de Gestión y venta de Componentes Electrónicos en línea

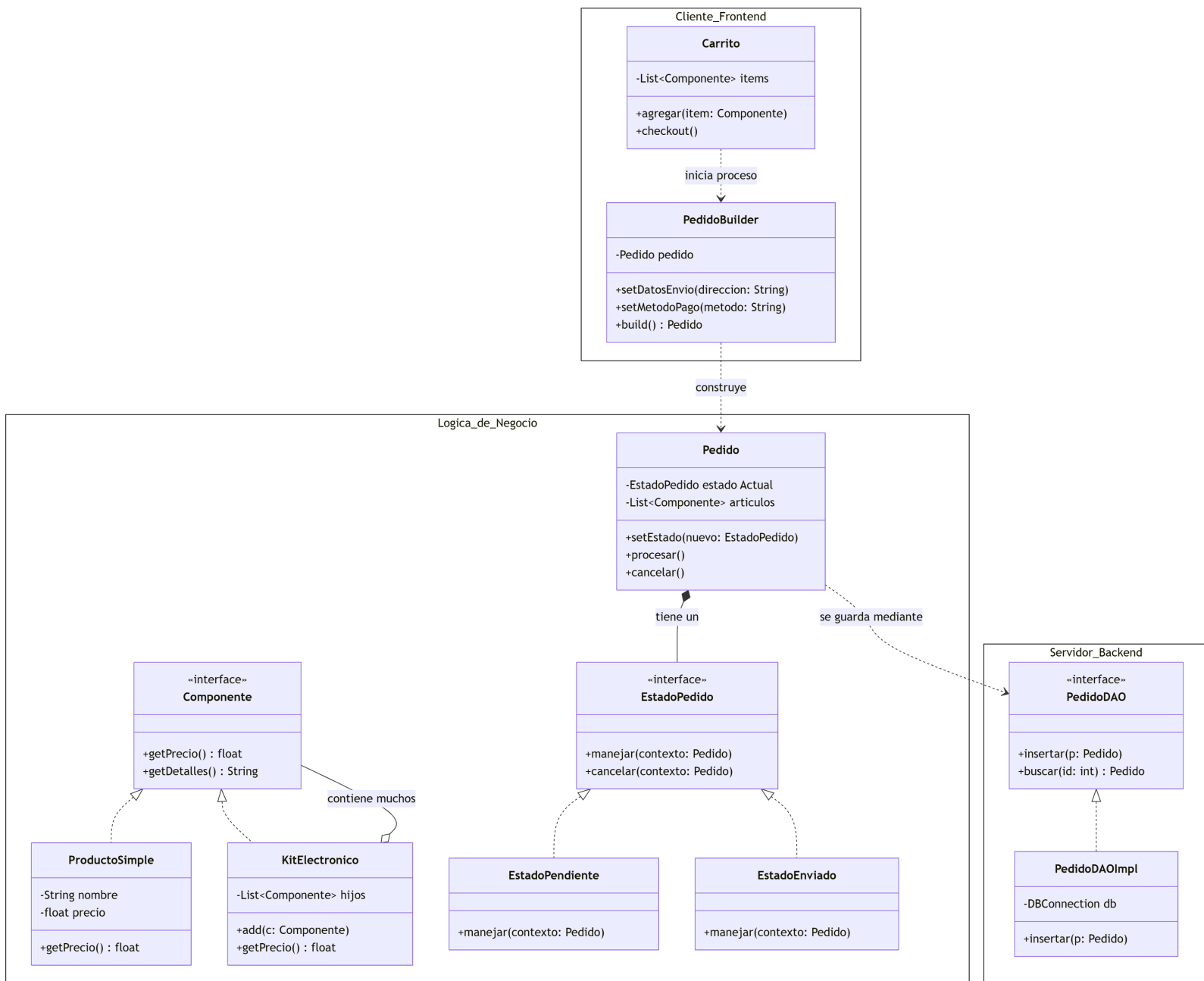
Pregunta: En el sistema de gestión de electrónica, se decide implementar un nuevo 'Kit de Robótica Avanzado' que permite al usuario elegir paso a paso qué sensores incluir mediante un asistente de configuración. Una vez pagado, el pedido entra en un estado de 'Validación Técnica' donde no se permiten cambios en los componentes pero sí en la dirección de envío.

Explique detalladamente cómo interactúan el Builder y el Composite en el cliente para armar este kit personalizado, y cómo el patrón State en el servidor debe gestionar las restricciones de modificación sin romper la encapsulación del DAO.

1. Descripción del problema

El sistema debe permitir a los clientes gestionar pedidos de componentes electrónicos individuales y también kits preconfigurados como kits de Arduino o robótica. Debido a la complejidad de algunos productos y sus estados, el sistema requiere una arquitectura que pueda separar la interfaz del usuario de la lógica de negocio y la persistencia de los datos. Esto garantiza escalabilidad e integridad de existencias, junto con una gestión fluida de pedidos.

2. Diagrama General UML



3. Ubicación explícita de cada patrón dentro de la arquitectura

Lado del Cliente (Frontend)

Encargado de la interacción directa con el usuario y la preparación de los datos antes de ser procesados por el servidor.

Patrón Composite: Se ubica en el catálogo y carrito de compras. Permite que la interfaz trate de forma idéntica a un componente simple y a un kit complejo, facilitando el cálculo de precios y la visualización jerárquica en la pantalla del usuario.

Patrón Builder: Se encuentra en el proceso de Checkout. Es el encargado de recolectar paso a paso la información del cliente para construir el objeto Pedido final de manera validada antes de enviarlo al servidor.

Lado del Servidor (Backend)

Encargado de la lógica de control, las reglas de negocio y la gestión de la persistencia de los componentes.

Patrón State: Se ubica en el núcleo de la lógica de negocio Domain Layer. Gestiona el ciclo de vida del pedido una vez recibido. Controla qué acciones son válidas y dispara notificaciones automáticas al cambiar de fase.

Patrón DAO: Se sitúa en la capa de acceso a datos. Actúa como el mediador entre el servidor y la base de datos de inventario. Su función es aislar la lógica del sistema de los detalles técnicos de SQL o el motor de base de datos utilizado.

4. Explicación del flujo del sistema.

I. Configuración y Selección

- **Uso de Composite:** El cliente navega entre componentes y Kits. El sistema trata a ambos como un objeto Componente, permitiendo que el carrito calcule el total sumando precios sin distinguir si es un ítem único o un paquete.
- **Uso de Builder:** Una vez que el usuario decide proceder al pago, el PedidoBuilder solicita paso a paso la dirección de entrega y el método de pago, construyendo un objeto Pedido complejo y validado antes de cualquier envío de datos.

II. Transmisión y Persistencia

- Uso de DAO: El objeto Pedido construido llega al servidor, donde el PedidoDAO recibe la orden. El DAO se encarga de traducir este objeto a una sentencia de base de datos para registrar la venta y descontar las existencias del inventario, aislando la lógica de negocio del motor de base de datos.

III. Gestión del Ciclo de Vida

- Uso de State: El Pedido inicia en un Estado Pendiente. Si el pago es validado, el servidor cambia internamente el objeto al Estado Empacado.
- Control de Reglas: Si el usuario intenta cancelar el pedido desde su aplicación, el objeto Estado actual verificará si la acción es válida; por ejemplo, si el estado ya cambió a Enviado, el patrón bloqueará la cancelación automáticamente.

IV. Finalización y Seguimiento

- El flujo termina cuando el servidor a través del DAO, actualiza el registro final a Entregado.
- El Cliente realiza consultas periódicas al Servidor para obtener el estado actual y mostrar al usuario el progreso en tiempo real.

5. Justificación de cómo los principios de diseño fortalecen la solución.

La integración de estos patrones bajo una arquitectura Cliente-Servidor permite que el sistema de gestión de componentes electrónicos sea altamente escalable y organizado. Al utilizar el principio de Responsabilidad Única, logramos que cada componente del sistema tenga una misión clara: mientras el Builder se encarga de la creación impecable de los pedidos en el cliente, el DAO se ocupa exclusivamente de que esos datos lleguen de forma segura a la base de datos en el servidor. Esta separación evita que errores en la interfaz afecten la integridad de los datos de inventario o las reglas de negocio principales.

Por otro lado, la aplicación mediante los patrones State y Composite otorga al sistema una flexibilidad necesaria para el mercado actual. Podemos introducir nuevos kits de robótica o añadir estados de envío adicionales sin necesidad de reescribir la lógica existente del carrito de compras o

del motor de pedidos. Esto reduce significativamente el riesgo de introducir errores al actualizar el sistema y permite que el backend evolucione de manera independiente al frontend.

Finalmente, el uso de la Inversión de Dependencias y el Bajo Acoplamiento a través del DAO y el modelo Cliente-Servidor asegura que el sistema no sea frágil ante cambios tecnológicos. Al depender de abstracciones e interfaces, el negocio no queda amarrado a una base de datos específica o a un protocolo de comunicación rígido. Esto fortalece la solución al permitir que el sistema crezca en usuarios y volumen de datos sin perder la claridad en el código ni la eficiencia en las respuestas hacia el usuario final.